



CODE SPORT

In this month's column, we feature a bunch of questions on operating systems and computer architecture.

Given that we have a number of requests from our student readers who are preparing for the on-campus interviews, let's discuss a few interview questions in this month's column. Since we had covered a number of questions on data structures and algorithms in last month's column, let's now focus on operating systems, computer architecture and some general CS concepts in this column.

1. We all know that when a process switches from user mode to kernel mode, the switch can be triggered by means of device interrupts, exceptions or software interrupts. Can you tell me what causes the switch back of the process from kernel mode to user mode?
2. Why is it that, typically, operating systems of the UNIX/Linux variety have two different data structures associated with a process, namely, *proc* structure or *u-area*? Is it necessary to have them separate? Why can't we have a single structure that can represent the information in both structures?
3. What is the difference between file systems that are *allocate on write* vs *allocate on update*? What are the advantages and disadvantages of each? For example, can you classify the following file systems as one of the above: (a) ZFS (b) NTFS (c) Ext4
4. Consider the following fragment illustrating the use of the *fork* system call:

```
pid_t mypid, childpid;
int status;
mypid = getpid();
printf("I am the parent process with pid %d.\n",
mypid);
childpid = fork();
```

```
if (childpid == 0) {
    printf("I am the child process);
    /* do something */
}
else
{
    Printf("in parent process \n");
    // wait(&status);
}
```

In the code fragment above, which describes a simple fork mechanism in UNIX, note that the *wait* call in the parent process has been commented out in the code. Assume that this was an inadvertent mistake done by the programmer and this code fragment has been invoked a number of times, say, about 1 million times. Can you explain how it would impact the operating system? *Hint*: Think in terms of the number of *proc* structures available.

5. Consider the following code fragment:

```
If (!access(file,...))
{
    f = fopen(file,"w+");
    foo(f);
    ...
}
else {
    fprintf(stderr, "Unable to open file %s.\n", file);
}
```

Can you identify the security vulnerability associated with the above code fragment? How would you overcome it?

6. Is it possible for the user to define a signal handler for the signal *KILL* in UNIX? If possible, explain how it can be done? If not, why is it not possible?